

ДИСЦИПЛИНА	Программирование в операционной системе Линукс
ИНСТИТУТ	Передовая инженерная школа СВЧ-электроники
КАФЕДРА	Передовых технологий
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания по дисциплине
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

Ссылка на материал:

<https://github.com/astafiev-rustam/programming-in-the-linux-operating-system/tree/lecture-1-7>

Лекция №7: Системное программирование: работа с файлами и процессами

Теоретическая вводная

Системное программирование: когда код встречается с операционной системой

Системное программирование в Linux — это искусство прямого общения с ядром операционной системы, минуя высокоуровневые абстракции. Представьте, что обычное программирование — это вождение автомобиля с автоматической коробкой передач: вы нажимаете педали, поворачиваете руль, и машина делает всю сложную работу за вас. Системное программирование — это управление гоночным болидом с механической коробкой: вы сами контролируете каждую передачу, обороты двигателя, момент сцепления, получая полный контроль над поведением машины, но и принимая на себя всю ответственность за каждое действие.

В основе этого подхода лежат системные вызовы — специальные функции, которые предоставляет ядро Linux для взаимодействия с аппаратными ресурсами. Когда ваша программа хочет создать файл, запустить процесс или выделить память, она в конечном счете обращается к ядру через эти вызовы. В отличие от стандартных библиотечных функций, которые могут быть переносимы между разными операционными системами, системные вызовы тесно связаны с конкретной ОС и предоставляют самый низкоуровневый и эффективный интерфейс для работы с системой.

Файловые дескрипторы: универсальные пропуск в мир ввода-вывода

В Linux всё есть файл — этот принцип пронизывает всю архитектуру системы. Но что это на самом деле означает? Речь идет не только о текстовых документах и исполняемых программах. Сокеты сетевых соединений, устройства вроде клавиатуры и мыши, каналы межпроцессного взаимодействия — все они представлены как файлы. И ключом к доступу ко всем этим ресурсам являются файловые дескрипторы.

Файловый дескриптор — это просто число, которое выдает ядро при открытии файла, сокета или другого ресурса. Но за этим числом скрывается мощная абстракция. Дескрипторы 0, 1 и 2 зарезервированы для стандартного ввода (stdin), вывода (stdout) и ошибок (stderr) соответственно. Когда вы открываете файл, ядро возвращает следующий свободный дескриптор (обычно 3), и через

этот номер вы можете читать и писать данные, управлять позицией в файле и выполнять другие операции.

Низкоуровневые функции вроде `open()`, `read()`, `write()` и `close()` работают непосредственно с этими дескрипторами, предоставляя полный контроль над операциями ввода-вывода. В отличие от высокоуровневых функций из стандартной библиотеки C, они не буферизуют данные (если явно не попросить), что делает их идеальными для ситуаций, где важны производительность и точное управление.

Процессы: независимые вселенные выполнения

Каждая программа в Linux работает в своем собственном процессе — изолированном окружении, которое ядро защищает от вмешательства других процессов. Представьте, что каждый процесс — это отдельная комната в большом отеле: у каждой свои окна (ввод-вывод), своя мебель (память), свои жильцы (потoki выполнения), и стены между комнатами обеспечивают приватность и безопасность.

Когда вы запускаете программу, ядро создает новый процесс, копируя структуру процесса-родителя. Но простое копирование было бы неэффективно, особенно если новая программа сразу же заменяет себя другой с помощью `exec()`. Поэтому Linux использует механизм *cory-on-write*: память копируется не сразу, а только когда один из процессов пытается изменить общие данные. Это оптимизация, которая демонстрирует, насколько глубоко продумана архитектура системы.

Функция `fork()` — это волшебный портал, который создает почти идентичную копию текущего процесса. "Почти" — потому что копия получает свой собственный PID и некоторые другие атрибуты, но наследует открытые файловые дескрипторы, переменные окружения и другую контекстную информацию. А `exec()` — это перерождение: процесс заменяет свою программу на совершенно другую, сохраняя при этом свой PID и некоторые другие характеристики.

Межпроцессное взаимодействие: искусство общения между мирами

Процессы, будучи изолированными, иногда нуждаются в общении друг с другом. Linux предоставляет богатый арсенал средств для межпроцессного взаимодействия (IPC). Самый простой из них — анонимные каналы (*pipes*), которые создаются с помощью системного вызова `pipe()`. Канал — это однонаправленный канал связи: что записано в один конец, может быть прочитано из другого.

Каналы идеально подходят для соединения вывода одной программы со входом другой — именно это происходит, когда вы используете символ `|` в командной строке. Но их возможности на этом не заканчиваются. Комбинируя `fork()` и `pipe()`, можно создавать сложные цепочки обработки данных, где каждый процесс выполняет свою специализированную задачу, передавая результаты следующему в конвейере.

Более сложные механизмы IPC, такие как именованные каналы (FIFO), разделяемая память и семафоры, предоставляют еще больше возможностей для взаимодействия между процессами. Но даже простые анонимные каналы, правильно использованные, могут решать *astonishingly* сложные задачи координации и передачи данных.

Сигналы: асинхронные прерывания для процессов

Сигналы — это способ, которым ядро и процессы могут асинхронно прерывать выполнение друг друга. Представьте, что вы работаете в офисе, и кто-то стучится в дверь — это сигнал. Вы можете решить

немедленно ответить на стук (обработать сигнал), отложить реакцию на потом или вообще проигнорировать его.

В Linux есть множество стандартных сигналов: SIGTERM для вежливого запроса на завершение, SIGKILL для немедленного "убийства", SIGINT для прерывания с клавиатуры (Ctrl+C), SIGHUP для уведомления о "отключении терминала". Процессы могут перехватывать большинство сигналов и реагировать на них осмысленным образом — сохранять данные, закрывать соединения, перечитывать конфигурационные файлы.

Умение работать с сигналами — важный навык системного программиста. Это позволяет создавать программы, которые корректно реагируют на внешние события, gracefully завершаются при получении команды на остановку и в целом ведут себя как "хорошие граждане" в экосистеме операционной системы.

Практические примеры

Пример 1: Низкоуровневая работа с файлами через файловые дескрипторы

Цель: Научиться использовать системные вызовы для работы с файлами.

```
# 1. Создаем программу для демонстрации низкоуровневых операций с файлами
cat > file_operations.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/stat.h>

int main() {
    int fd;
    ssize_t bytes;
    char buffer[100];

    printf("=== Низкоуровневая работа с файлами ===\n\n");

    // 1. Создаем и открываем файл для записи (O_CREAT | O_WRONLY)
    // 0644 - права доступа: владелец читает/пишет, остальные только читают
    fd = open("test_file.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd == -1) {
        perror("Ошибка при создании файла");
        exit(1);
    }
    printf("1. Файл создан, дескриптор: %d\n", fd);

    // 2. Записываем данные в файл
    char* data = "Привет, системное программирование!\n";
    bytes = write(fd, data, strlen(data));
    printf("2. Записано %zd байт в файл\n", bytes);
```

```
// 3. Закрываем файл
close(fd);
printf("3. Файл закрыт\n");

// 4. Открываем файл для чтения
fd = open("test_file.txt", O_RDONLY);
if (fd == -1) {
    perror("Ошибка при открытии файла");
    exit(1);
}
printf("4. Файл открыт для чтения, дескриптор: %d\n", fd);

// 5. Читаем данные из файла
bytes = read(fd, buffer, sizeof(buffer) - 1);
if (bytes == -1) {
    perror("Ошибка при чтении файла");
    close(fd);
    exit(1);
}
buffer[bytes] = '\0'; // Добавляем нулевой терминатор
printf("5. Прочитано %zd байт:\n%s\n", bytes, buffer);

// 6. Получаем информацию о файле
struct stat file_info;
if (fstat(fd, &file_info) == 0) {
    printf("6. Информация о файле:\n");
    printf("    Размер: %ld байт\n", file_info.st_size);
    printf("    Inode: %ld\n", file_info.st_ino);
    printf("    Права доступа: %o\n", file_info.st_mode & 0777);
}

// 7. Закрываем файл
close(fd);

// 8. Демонстрация работы с разными позициями в файле
fd = open("test_file.txt", O_RDWR);
lseek(fd, 8, SEEK_SET); // Перемещаемся на 8 байт от начала
write(fd, "LINUX", 5);   // Заменяем часть текста

lseek(fd, 0, SEEK_SET); // Возвращаемся в начало
bytes = read(fd, buffer, sizeof(buffer) - 1);
buffer[bytes] = '\0';
printf("7. После модификации: %s\n", buffer);

close(fd);

printf("\n=== Демонстрация завершена ===\n");
return 0;
}
EOF
```

```
# 2. Компилируем и запускаем программу
gcc file_operations.c -o file_operations
./file_operations
```

```
# 3. Проверяем созданный файл
echo -e "\n=== Содержимое созданного файла ==="
cat test_file.txt

# 4. Проверяем права доступа
echo -e "\n=== Права доступа к файлу ==="
ls -la test_file.txt
```

Пример 2: Создание процессов с fork() и управление ими

Цель: Освоить создание процессов и базовое управление ими.

```
# 1. Создаем программу для демонстрации работы с процессами
cat > process_demo.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main() {
    pid_t pid;
    int status;

    printf("=== Демонстрация работы с процессами ===\n\n");
    printf("Родительский процесс: PID=%d, PPID=%d\n", getpid(), getppid());

    // Создаем дочерний процесс
    pid = fork();

    if (pid == -1) {
        perror("Ошибка при создании процесса");
        exit(1);
    }
    else if (pid == 0) {
        // Код выполняется в дочернем процессе
        printf("Дочерний процесс: PID=%d, PPID=%d\n", getpid(), getppid());

        // Имитируем работу дочернего процесса
        for (int i = 0; i < 3; i++) {
            printf("Дочерний процесс: работаю... (%d/3)\n", i + 1);
            sleep(1);
        }

        printf("Дочерний процесс: завершаю работу\n");
        exit(42); // Завершаем с кодом 42
    }
    else {
        // Код выполняется в родительском процессе
        printf("Родительский процесс: создал дочерний процесс с PID=%d\n", pid);
```

```
    // Ждем завершения дочернего процесса
    printf("Родительский процесс: жду завершения дочернего процесса...\n");
    wait(&status);

    if (WIFEXITED(status)) {
        printf("Родительский процесс: дочерний процесс завершился с кодом
%d\n",
                WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status)) {
        printf("Родительский процесс: дочерний процесс убит сигналом %d\n",
                WTERMSIG(status));
    }

    printf("Родительский процесс: завершаю работу\n");
}

return 0;
}
EOF
```

```
# 2. Компилируем и запускаем
gcc process_demo.c -o process_demo
./process_demo
```

```
# 3. Создаем более сложный пример с несколькими процессами
```

```
cat > multiple_processes.c << 'EOF'
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
void worker_process(int id) {
    printf("Рабочий процесс %d (PID=%d) начал работу\n", id, getpid());
    sleep(id * 2); // Имитируем разную продолжительность работы
    printf("Рабочий процесс %d (PID=%d) завершил работу\n", id, getpid());
    exit(id * 10);
}
```

```
int main() {
    printf("=== Множественные процессы ===\n\n");
    printf("Главный процесс: PID=%d\n", getpid());
```

```
    const int NUM_WORKERS = 3;
    pid_t pids[NUM_WORKERS];
```

```
    // Создаем несколько дочерних процессов
    for (int i = 0; i < NUM_WORKERS; i++) {
        pids[i] = fork();
```

```
        if (pids[i] == 0) {
            // Дочерний процесс
            worker_process(i + 1);
        }
    }
}
```

```

        // Функция worker_process вызывает exit(), так что сюда мы не попадем
    }
    else if (pids[i] == -1) {
        perror("Ошибка при создании процесса");
        exit(1);
    }
    else {
        printf("Создан рабочий процесс %d с PID=%d\n", i + 1, pids[i]);
    }
}

// Родительский процесс ждет завершения всех дочерних
printf("\nГлавный процесс: жду завершения всех рабочих процессов...\n");

for (int i = 0; i < NUM_WORKERS; i++) {
    int status;
    pid_t finished_pid = wait(&status);

    // Находим, какой это был рабочий процесс
    int worker_id = -1;
    for (int j = 0; j < NUM_WORKERS; j++) {
        if (pids[j] == finished_pid) {
            worker_id = j + 1;
            break;
        }
    }

    if (WIFEXITED(status)) {
        printf("Рабочий процесс %d (PID=%d) завершился с кодом %d\n",
            worker_id, finished_pid, WEXITSTATUS(status));
    }
}

printf("Главный процесс: все рабочие процессы завершены\n");
return 0;
}
EOF

```

```

# 4. Компилируем и запускаем
gcc multiple_processes.c -o multiple_processes
./multiple_processes

```

Пример 3: Межпроцессное взаимодействие через каналы (pipes)

Цель: Научиться использовать каналы для обмена данными между процессами.

```

# 1. Создаем программу с использованием каналов
cat > pipe_demo.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

```

```
#include <string.h>
#include <sys/wait.h>

#define BUFFER_SIZE 1024

int main() {
    int pipefd[2]; // pipefd[0] - чтение, pipefd[1] - запись
    pid_t pid;
    char buffer[BUFFER_SIZE];
    ssize_t bytes;

    printf("=== Межпроцессное взаимодействие через каналы ===\n\n");

    // Создаем канал
    if (pipe(pipefd) == -1) {
        perror("Ошибка при создании канала");
        exit(1);
    }

    printf("Канал создан: чтение=%d, запись=%d\n", pipefd[0], pipefd[1]);

    // Создаем дочерний процесс
    pid = fork();

    if (pid == -1) {
        perror("Ошибка при создании процесса");
        exit(1);
    }

    if (pid == 0) {
        // Дочерний процесс - читает из канала
        close(pipefd[1]); // Закрываем конец для записи

        printf("Дочерний процесс (PID=%d): жду данные из канала...\n", getpid());

        bytes = read(pipefd[0], buffer, BUFFER_SIZE - 1);
        if (bytes == -1) {
            perror("Ошибка при чтении из канала");
            exit(1);
        }

        buffer[bytes] = '\0';
        printf("Дочерний процесс: получил %zd байт: '%s'\n", bytes, buffer);

        // Отправляем ответ обратно (для этого нужен был бы второй канал)
        printf("Дочерний процесс: завершаю работу\n");
        close(pipefd[0]);
        exit(0);
    } else {
        // Родительский процесс - пишет в канал
        close(pipefd[0]); // Закрываем конец для чтения

        sleep(1); // Даем дочернему процессу время подготовиться
```



```
char* message = "Привет из родительского процесса!";
printf("Родительский процесс (PID=%d): отправляю сообщение...\n",
getpid());

bytes = write(pipefd[1], message, strlen(message));
if (bytes == -1) {
    perror("Ошибка при записи в канал");
    exit(1);
}

printf("Родительский процесс: отправил %zd байт\n", bytes);
close(pipefd[1]);

// Ждем завершения дочернего процесса
wait(NULL);
printf("Родительский процесс: дочерний процесс завершился\n");
}

return 0;
}
EOF
```

```
# 2. Компилируем и запускаем
gcc pipe_demo.c -o pipe_demo
./pipe_demo
```

```
# 3. Создаем более сложный пример с двунаправленной связью
```

```
cat > bidirectional_pipe.c << 'EOF'
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

#define BUFFER_SIZE 1024

int main() {
    int parent_to_child[2]; // Канал от родителя к ребенку
    int child_to_parent[2]; // Канал от ребенка к родителю
    pid_t pid;
    char buffer[BUFFER_SIZE];

    printf("=== Двунаправленная связь между процессами ===\n\n");

    // Создаем оба канала
    if (pipe(parent_to_child) == -1 || pipe(child_to_parent) == -1) {
        perror("Ошибка при создании каналов");
        exit(1);
    }

    pid = fork();

    if (pid == -1) {
```

```
    perror("Ошибка при создании процесса");
    exit(1);
}

if (pid == 0) {
    // Дочерний процесс
    close(parent_to_child[1]); // Закрываем запись в первый канал
    close(child_to_parent[0]); // Закрываем чтение из второго канала

    // Читаем сообщение от родителя
    ssize_t bytes = read(parent_to_child[0], buffer, BUFFER_SIZE - 1);
    buffer[bytes] = '\0';
    printf("Дочерний процесс: получил - '%s'\n", buffer);

    // Отправляем ответ
    char* response = "Привет, родитель! Я получил твое сообщение.";
    write(child_to_parent[1], response, strlen(response));
    printf("Дочерний процесс: отправил ответ\n");

    close(parent_to_child[0]);
    close(child_to_parent[1]);
    exit(0);
} else {
    // Родительский процесс
    close(parent_to_child[0]); // Закрываем чтение из первого канала
    close(child_to_parent[1]); // Закрываем запись во второй канал

    // Отправляем сообщение ребенку
    char* message = "Привет, дочерний процесс! Как дела?";
    printf("Родительский процесс: отправляю - '%s'\n", message);
    write(parent_to_child[1], message, strlen(message));

    // Читаем ответ
    ssize_t bytes = read(child_to_parent[0], buffer, BUFFER_SIZE - 1);
    buffer[bytes] = '\0';
    printf("Родительский процесс: получил ответ - '%s'\n", buffer);

    close(parent_to_child[1]);
    close(child_to_parent[0]);
    wait(NULL);
    printf("Родительский процесс: обмен завершен\n");
}

return 0;
}
EOF
```

4. Компилируем и запускаем

```
gcc bidirectional_pipe.c -o bidirectional_pipe
./bidirectional_pipe
```

Пример 4: Запуск внешних программ с `exec()`

Цель: Научиться заменять программу процесса на другую с помощью `exec`.

```
# 1. Создаем программу для демонстрации exec
cat > exec_demo.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    pid_t pid;

    printf("=== Запуск внешних программ с exec() ===\n\n");
    printf("Родительский процесс: PID=%d\n", getpid());

    pid = fork();

    if (pid == -1) {
        perror("Ошибка при создании процесса");
        exit(1);
    }

    if (pid == 0) {
        // Дочерний процесс
        printf("Дочерний процесс: PID=%d\n", getpid());
        printf("Дочерний процесс: запускаю программу 'ls'...\n\n");

        // Заменяем программу дочернего процесса на 'ls'
        execl("/bin/ls", "ls", "-la", NULL);

        // Если execl вернул управление, значит произошла ошибка
        perror("Ошибка при выполнении execl");
        exit(1);
    } else {
        // Родительский процесс
        wait(NULL);
        printf("\nРодительский процесс: дочерний процесс завершился\n");
    }

    // Демонстрация разных вариантов exec
    printf("\n--- Демонстрация разных функций exec ---\n");

    pid = fork();
    if (pid == 0) {
        printf("\nДочерний процесс: запускаю 'ps' с аргументами...\n");

        // Используем execvp с массивом аргументов
        char* args[] = {"ps", "aux", NULL};
```

```
    execvp("ps", args);

    perror("Ошибка при выполнении execvp");
    exit(1);
} else {
    wait(NULL);
}

pid = fork();
if (pid == 0) {
    printf("\nДочерний процесс: запускаю 'pwd'...\n");

    // Используем execlp (ищет в PATH)
    execlp("pwd", "pwd", NULL);

    perror("Ошибка при выполнении execlp");
    exit(1);
} else {
    wait(NULL);
}

printf("\nРодительский процесс: все демонстрации завершены\n");
return 0;
}
EOF
```

```
# 2. Компилируем и запускаем
gcc exec_demo.c -o exec_demo
./exec_demo
```

```
# 3. Создаем пример с комбинацией fork + exec + pipe
cat > fork_exec_pipe.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define BUFFER_SIZE 1024

int main() {
    int pipefd[2];
    pid_t pid;
    char buffer[BUFFER_SIZE];
    ssize_t bytes;

    printf("=== Комбинация: fork + exec + pipe ===\n\n");

    // Создаем канал
    if (pipe(pipefd) == -1) {
        perror("Ошибка при создании канала");
        exit(1);
    }
```

```

pid = fork();

if (pid == 0) {
    // Дочерний процесс - будет выполнять 'ls'
    close(pipefd[0]); // Закрываем чтение

    // Перенаправляем stdout в канал
    dup2(pipefd[1], STDOUT_FILENO);
    close(pipefd[1]);

    // Запускаем ls
    execlp("ls", "ls", "-la", NULL);
    perror("Ошибка при выполнении execlp");
    exit(1);
} else {
    // Родительский процесс
    close(pipefd[1]); // Закрываем запись

    printf("Родительский процесс: читаю вывод команды 'ls -la':\n");
    printf("=====\n");

    // Читаем вывод команды ls из канала
    while ((bytes = read(pipefd[0], buffer, BUFFER_SIZE - 1)) > 0) {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    printf("=====\n");

    close(pipefd[0]);
    wait(NULL);
    printf("Родительский процесс: завершено\n");
}

return 0;
}
EOF

# 4. Компилируем и запускаем
gcc fork_exec_pipe.c -o fork_exec_pipe
./fork_exec_pipe

```

Пример 5: Работа с сигналами и обработка прерываний

Цель: Научиться обрабатывать сигналы и корректно завершать программы.

```

# 1. Создаем программу для демонстрации работы с сигналами
cat > signal_demo.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>

```

```
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t keep_running = 1;
volatile sig_atomic_t signal_count = 0;

// Обработчик для SIGINT (Ctrl+C)
void handle_sigint(int sig) {
    signal_count++;
    printf("\nПолучен сигнал SIGINT (%d). Сигналов получено: %d\n",
           sig, signal_count);
    printf("Нажмите Ctrl+C еще раз для быстрого завершения или подождите 5 секунд\n");

    if (signal_count >= 2) {
        printf("Экстренное завершение!\n");
        exit(1);
    }

    // Устанавливаем таймер для сброса счетчика
    alarm(5);
}

// Обработчик для SIGALRM
void handle_sigalrm(int sig) {
    printf("Таймер истек, счетчик сигналов сброшен\n");
    signal_count = 0;
}

// Обработчик для SIGTERM
void handle_sigterm(int sig) {
    printf("\nПолучен сигнал SIGTERM (%d). Корректно завершаю работу...\n", sig);
    keep_running = 0;
}

// Обработчик для SIGUSR1 (пользовательский сигнал)
void handle_sigusr1(int sig) {
    printf("\nПолучен пользовательский сигнал SIGUSR1 (%d)\n", sig);
    printf("Текущее время: ");
    fflush(stdout);
    system("date");
}

int main() {
    printf("=== Демонстрация работы с сигналами ===\n\n");
    printf("Процесс PID=%d\n", getpid());
    printf("Используемые сигналы:\n");
    printf("  Ctrl+C (SIGINT) - попробуйте нажать 1 или 2 раза\n");
    printf("  SIGTERM - kill %d\n", getpid());
    printf("  SIGUSR1 - kill -USR1 %d\n", getpid());
    printf("  SIGALRM - внутренний таймер\n\n");

    // Регистрируем обработчики сигналов
```

```
signal(SIGINT, handle_sigint);
signal(SIGTERM, handle_sigterm);
signal(SIGUSR1, handle_sigusr1);
signal(SIGALRM, handle_sigalrm);

printf("Программа работает. Отправьте сигналы как указано выше.\n");
printf("Для выхода можно также подождать 30 секунд.\n\n");

int counter = 0;
while (keep_running && counter < 30) {
    printf("Работаю... (%d/30 секунд)\r", counter);
    fflush(stdout);
    sleep(1);
    counter++;
}

if (keep_running) {
    printf("\nЗавершение по таймауту\n");
}

printf("Программа корректно завершена\n");
return 0;
}
EOF
```

2. Компилируем и запускаем в фоне

```
gcc signal_demo.c -o signal_demo
./signal_demo &
```

3. Запоминаем PID процесса

```
SIGNAL_PID=$!
echo "Запущен процесс с PID: $SIGNAL_PID"
```

4. Тестируем отправку сигналов

```
sleep 2
echo -e "\n=== Тестируем SIGUSR1 ==="
kill -USR1 $SIGNAL_PID
```

```
sleep 2
echo -e "\n=== Тестируем Ctrl+C (в другом терминале) ==="
echo "Нажмите Ctrl+C в терминале где запущен signal_demo"
```

```
sleep 5
echo -e "\n=== Тестируем SIGTERM ==="
kill -TERM $SIGNAL_PID
```

5. Ждем завершения

```
wait $SIGNAL_PID
echo "Процесс завершен"
```

6. Создаем демон-процесс

```
cat > daemon_demo.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>
#include <signal.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <time.h>

volatile sig_atomic_t daemon_running = 1;

void handle_shutdown(int sig) {
    daemon_running = 0;
}

int main() {
    printf("Запуск демона...\n");

    // 1. Создаем дочерний процесс
    pid_t pid = fork();

    if (pid < 0) {
        perror("Ошибка при fork");
        exit(1);
    }

    if (pid > 0) {
        // Родительский процесс завершается
        printf("Демон запущен с PID: %d\n", pid);
        exit(0);
    }

    // 2. Создаем новую сессию (отсоединяем от терминала)
    if (setsid() < 0) {
        perror("Ошибка при setsid");
        exit(1);
    }

    // 3. Устанавливаем обработчики сигналов
    signal(SIGTERM, handle_shutdown);
    signal(SIGINT, handle_shutdown);

    // 4. Закрываем стандартные дескрипторы
    close(STDIN_FILENO);
    close(STDOUT_FILENO);
    close(STDERR_FILENO);

    // 5. Открываем log файл
    int log_fd = open("/tmp/daemon.log", O_CREAT | O_WRONLY | O_APPEND, 0644);
    if (log_fd < 0) {
        exit(1);
    }

    // 6. Демон работает
    while (daemon_running) {
        time_t now = time(NULL);
        char* time_str = ctime(&now);
```



```
    // Пишем в log
    dprintf(log_fd, "Демон работает: %s", time_str);
    fsync(log_fd);

    sleep(5);
}

// 7. Корректное завершение
dprintf(log_fd, "Демон завершает работу\n");
close(log_fd);

return 0;
}
EOF

# 7. Компилируем и тестируем демона
gcc daemon_demo.c -o daemon_demo
echo -e "\n=== Запуск демона ==="
./daemon_demo

# Даем демону поработать
sleep 3
echo -e "\n=== Проверяем log файл ==="
cat /tmp/daemon.log

# Завершаем демона
DAEMON_PID=$(ps aux | grep daemon_demo | grep -v grep | awk '{print $2}')
if [ ! -z "$DAEMON_PID" ]; then
    echo "Завершаем демона с PID: $DAEMON_PID"
    kill $DAEMON_PID
fi
```